Intelligent Manufacturing Quality Control

Implementation, training, inference and release lifecycle · Repository review: October 2026

Scope. A locally executed visual-inspection system for six known steel surface-pattern classes. The frontend supports inspection uploads, human review, model evaluation, activation and monitoring. This is not an automated industrial accept/reject system.

APPLICATION AND MODEL PATH

OFFLINE PIPELINE

TensorFlow

Validated images, reproducible splits, held-out evaluation, checkpoint selection and export to ONNX opset 17.

ONLINE API

Rust / Axum

Cached ONNX Runtime session, bounded uploads, shared pixel preprocessing and saved predictions.

EVIDENCE STORE

Firestore

Inspections, corrections, audit events and release pointers; local/GCS object adapter for images and weights.

MODEL CONTRACT

Six classes: crazing, inclusion, patches, pitted surface, rolled-in scale and scratches.
Input: grayscale 64 × 64; deterministic resize, integer luminance and NHWC float32 normalization.
CNN: three convolution layers, global pooling, dense layers and six-way softmax.

Training: TensorFlow 2.16.2 and Keras; stratified and deduplicated splits, checkpointing and independent test metrics.
Export: ONNX opset 17, executable graph and output checks, TensorFlow / ONNX numerical comparison.
Serving: model loaded once and executed in a Rust blocking worker.

MODEL RELEASE AND REVIEW

Candidate

Register an immutable package containing ONNX weights, dataset/split IDs, metrics and checksums.

Validate

Execute the artifact, enforce class-recall and macro-F1 thresholds, and reject incompatible comparisons.

Activate / rollback

Prepare a runtime session, commit active/previous pointers and history, then swap or restore the model.

Review

Preserve original probabilities and model version; append separate operator corrections and audit records.

Intelligent Manufacturing Quality Control

Recorded local verification, metrics, limitations and deployment boundaries

ACTUAL LOCAL VERIFICATION

21	36	4.77e-7
passed tests	**parity samples**	**maximum error**
6 Python, 9 Rust, 6 frontend	Full TF / ONNX Runtime check	Absolute probability difference

TRAINING RESULTS — PROCEDURAL SMOKE DATA ONLY

Run	Held-out accuracy	Macro-F1	Context
smoke-v1	0.667	0.556	Gate failures
smoke-a	1.000	1.000	Synthetic, 36 test
smoke-b	1.000	1.000	Synthetic, 36 test

The complete dataset contains 216 project-owned procedural images (36 per class) with 150 training, 30 validation and 36 held-out test images. A perfect synthetic score demonstrates a functioning pipeline only; no industrial classification accuracy is established.

EXECUTED TESTS

Verified locally

Python dataset and artifact tests, Rust unit and actual ONNX fixture tests, frontend workflow tests, formatter and linter checks, production builds, Firestore emulator inspection/review/release/rollback, container restart restoration and persisted browser review.

LOCAL BENCHMARK

Limited performance sample

Twenty requests with four workers against a repeated procedural image: 100.59 requests/s, 37.46 ms mean and 87.22 ms p95 end-to-end. This is not sustained factory throughput or a production latency SLA.

UNVERIFIED AND OUT OF SCOPE

Real NEU-CLS training and rights verification; representative industrial precision/recall; remote GitHub Actions runs; GCP IAM, GCS, Cloud Run and production Firestore integration; distributed release coordination; long-duration/fault-injection testing; automated retention; full browser file-chooser testing. Local operator-token access is not a role-based production identity system.

REPRODUCTION

The repository includes docs/version1-report.md, docs/testing.md and docs/verification/results.json, plus trained ONNX CI fixtures. Run the local Firestore emulator, use scripts/bootstrap-smoke.sh to prepare procedural models, then execute scripts/check.sh and scripts/api-smoke.py as documented in docs/setup.md.